SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: OPERATING SYSTEMS **COURSE CODE: CSA0416**

NAME:PREETHI R **REG NO:192511352**

COURSE OUTCOME COVERED

CO2: Analyze process management and deadlock handling techniques to improve CPU utilization and system performance(BL3-BL4)

Assessment Tool Weightage: 30%

QUESTIONS: 10 **Total Marks:100**

Annexure A

Scenario based

Code of Conduct:

I __ PREETHI R/12511352____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Question 1 — CPU Scheduling in a Real-Time Ticket Booking System

A railway ticket booking system experiences heavy load during festival seasons. The system handles three categories of processes:

- P1 – Payment gateway transactions (High priority)
- P2 – Seat availability checks (Medium priority)
- P3 – Ticket confirmation and printing (Low priority)
- Burst times vary from 2 ms to 20 ms.

Recently, customers complained that ticket confirmations are taking too long.

Tasks

1. Identify the most suitable scheduling algorithm (FCFS, SJF, Priority, RR) for this scenario.
2. Justify *why* your chosen method improves CPU utilization and reduces customer wait time.
3. Provide a sample execution order for the above categories.
4. Discuss one drawback of this scheduling method in real-time systems.

(10 Marks)

Question 2 — Deadlock in a Banking Transaction Server

A bank server handles the following operations simultaneously:

- T1: Transfers money between accounts
- T2: Updates passbook entries
- T3: Generates monthly statements

Due to simultaneous locking of Account Database (A), Customer Info Database (B), and Transaction Logs (C), the system enters a state where:

- T1 holds A and requests B
- T2 holds B and requests C
- T3 holds C and requests A

Tasks

1. Draw the resource request situation (text-based RAG).
2. Identify whether a deadlock has occurred and justify your answer.

3. Recommend *one specific technique* (avoidance/prevention/recovery) and explain how it fixes the issue.
4. Suggest a redesign for the locking order to avoid future deadlocks.

(10 Marks)

Question 3 — Mobile OS App Switching Lag

A mobile operating system is designed to support smooth app switching.

But users report that:

- Opening WhatsApp delays the background music player.
- Switching between social media apps feels laggy.
- CPU shows high idle times despite multiple running apps.

The OS currently uses Round Robin (quantum = 20 ms) with 20 background apps active.

Tasks

1. Analyze why the CPU remains idle despite many active processes.
2. Determine whether the time quantum is helping or hurting performance.
3. Recommend a better scheduling method or quantum value, with justification.
4. Explain how the change will reduce switching lag and improve system throughput.

(10 Marks)

Question 4 — Deadlock Risk in an Operating System Update

During a Windows OS update, several modules run in parallel:

- M1: Copy update files
- M2: Install drivers
- M3: Update registry
- M4: Validate system integrity

Resources involved:

- R1 = Disk I/O
- R2 = Kernel lock
- R3 = Configuration lock

Recently logs show:

- M1 holds R1, waiting for R2
- M2 holds R2, waiting for R3

- M3 holds R3, waiting for R1

Tasks

1. Identify whether this sequence can lead to a deadlock.
2. If yes, explain the exact circular wait condition.
3. Propose *Banker's algorithm style* resource allocation to avoid risk.
4. Suggest how OS designers can prioritize modules to improve CPU utilization during update.

(10 Marks)

Question 5 — Cloud Server CPU Utilization Analysis

A cloud hosting provider notices:

- 80% of the time, VM processes wait for network or disk
- Only 20% of the time is spent on CPU
- Each server runs 7 VMs

Using the formula:

$$U = 1 - p^n$$

where p = I/O wait probability

Tasks

1. Calculate CPU utilization.
2. Interpret whether the server is underutilized or optimally loaded.
3. Suggest how increasing or decreasing number of VMs affects throughput.
4. Propose two OS-level strategies for improving CPU utilization in this cloud environment.

(10 Marks)

Question 6 — Hospital Emergency System Process Scheduling

A hospital information system manages:
• P1 – Emergency patient registration (Highest Priority)
• P2 – Laboratory report generation (Medium Priority)
• P3 – Pharmacy billing (Low Priority)

During peak hours, pharmacy billing is delayed while emergency requests continue to arrive.

Tasks
1. Identify the scheduling issue occurring in the system.

2. Recommend a suitable CPU scheduling algorithm.
3. Explain how starvation can be prevented.
4. Discuss how your solution improves response time and fairness.

(10 Marks)

Question 7 — E-Commerce Warehouse Synchronization

An e-commerce warehouse has multiple worker threads updating a shared inventory database simultaneously.

Developers observe:
• Incorrect stock counts
• Duplicate order confirmations
• Frequent data inconsistencies

Tasks
1. Identify the synchronization problem.
2. Explain why a critical section is required.
3. Recommend either semaphores or monitors to solve the issue.
4. Explain how the proposed solution maintains data consistency.

(10 Marks)

Question 8 — Video Streaming Service Thread Management

A video streaming platform creates one thread for every client connection.

During a live sports event:
• 60,000 users connect simultaneously.
• CPU utilisation reaches 95%.
• Memory consumption increases rapidly.
• Video buffering becomes frequent.

Tasks
1. Identify the thread-management problem.
2. Recommend a suitable multithreading model.
3. Suggest two optimisations to improve scalability.
4. Explain how your recommendations improve throughput.

(10 Marks)

Question 9 — Smart Traffic Control Deadlock

A smart city traffic management system controls four intersections.

Resources:

• R1 – North Junction
• R2 – East Junction
• R3 – South Junction
• R4 – West Junction

Current situation:
• T1 holds R1 and requests R2.
• T2 holds R2 and requests R3.
• T3 holds R3 and requests R4.
• T4 holds R4 and requests R1.

Tasks
1. Determine whether a deadlock exists.
2. Explain the four necessary conditions for deadlock in this scenario.
3. Recommend one recovery technique.
4. Suggest a redesign that prevents similar deadlocks.

(10 Marks)

Question 10 — University Examination Server Resource Allocation

An online examination server allocates CPU, memory and database connections to student sessions.

At examination time:
• Thousands of students log in simultaneously.
• New resource requests arrive continuously.
• The administrator wants to avoid deadlocks without reducing resource utilisation.

Tasks
1. Explain how Banker's Algorithm can be applied.
2. Describe the information required before granting a request.
3. Explain how the system determines whether it is in a safe state.
4. Discuss one limitation of using Banker's Algorithm in large-scale cloud systems.

(10 Marks)

QUESTION 1: CPU SCHEDULING IN A REAL-TIME TICKET BOOKING SYSTEM (10 MARKS)

1. Identify the most suitable scheduling algorithm.

Priority Scheduling is the most suitable CPU scheduling algorithm for the railway ticket booking system because the system handles different categories of processes with different levels of importance. Payment gateway transactions (P1) have high priority because they involve financial operations and must be processed quickly. Seat availability checks (P2) have medium priority, while ticket confirmation and printing (P3) have low priority.

In Priority Scheduling, the CPU is allocated to the ready process with the highest priority. Therefore, payment transactions are processed before less critical tasks. Preemptive Priority Scheduling is particularly useful because a newly arriving high-priority process can interrupt a lower-priority process. This helps the system respond quickly to important booking operations during festival seasons.

2. Explain how the selected method improves CPU utilization and reduces customer waiting time.

Priority Scheduling reduces the response time of critical booking operations by allowing payment transactions to execute before lower-priority tasks. During festival seasons, many customers may attempt to book tickets simultaneously, so payment processing must be handled efficiently to avoid unnecessary delays.

CPU utilization can be improved when the operating system selects another ready process while the current process waits for an input/output operation. For example, if a payment transaction waits for a response from the payment gateway, the CPU can execute a seat availability check instead of remaining idle. This improves the use of available CPU resources.

However, Priority Scheduling does not always minimize the average waiting time of all processes. Its main purpose is to give preference to important operations.

3. Provide a sample execution order.

Assume that all three processes arrive at time 0 and each requires 5 ms of CPU time. The execution order is P1, P2, and P3 according to their priorities.

Time (ms):

0 5 10 15 |----- P1 -----|----- P2 -----|----- P3 -----|

Execution order: P1 -> P2 -> P3

P1 executes first because payment processing has the highest priority. After P1 completes, P2 checks seat availability. Finally, P3 confirms and prints the ticket. This example assumes that no new higher-priority process arrives during execution.

4. Discuss one drawback of this scheduling method in real-time systems.

One major drawback of Priority Scheduling is starvation. If high-priority payment transactions continue to arrive, the low-priority ticket confirmation process may remain in the ready queue for a long time. This increases the waiting time for customers who are waiting for their tickets to be confirmed.

This problem can be reduced using aging. In aging, the priority of a process gradually increases as its waiting time increases. As a result, low-priority processes eventually get an opportunity to execute. The system should also monitor the deadlines of important booking operations.

Conclusion:

Priority Scheduling is suitable for the ticket booking system because it gives preference to critical payment transactions and improves their response time. Using aging can reduce starvation and improve fairness among processes.

QUESTION 2: DEADLOCK IN A BANKING TRANSACTION SERVER (10 MARKS)

1. Draw the resource request situation using a text-based Resource Allocation Graph.

A Resource Allocation Graph (RAG) represents the relationship between transactions and resources. In the banking server, T1 transfers money between accounts, T2 updates passbook entries, and T3 generates monthly statements.

T1 holds the Account Database (A) and requests the Customer Info Database (B). T2 holds B and requests the Transaction Logs (C). T3 holds C and requests A.

The text-based Resource Allocation Graph is:

A -> T1 -> B -> T2 -> C -> T3 -> A

An arrow from a resource to a transaction represents resource allocation, while an arrow from a transaction to a resource represents a pending request. The graph shows that each transaction holds one resource and waits for another resource.

2. Identify whether a deadlock has occurred and justify your answer.

Yes, a deadlock has occurred, assuming that each resource is protected by an exclusive lock and cannot be released while its transaction is waiting.

T1 cannot proceed because resource B is held by T2. T2 cannot proceed because resource C is held by T3. T3 cannot proceed because resource A is held by T1. Therefore, each transaction waits for another transaction to release its resource, and none can continue.

The circular dependency is T1 waiting for T2, T2 waiting for T3, and T3 waiting for T1. This creates a circular wait condition. Since each resource has a single exclusive lock in this scenario, the cycle confirms the deadlock.

3. Recommend one specific technique and explain how it fixes the issue.

Deadlock Prevention through resource ordering is a suitable solution. In this method, the system assigns a fixed order to resources, such as A, B, and C. Every transaction must acquire resources according to this order.

For example, a transaction requiring resources A and C must acquire A before C. It must not acquire C first and then request A. This prevents circular wait because transactions cannot acquire resources in conflicting orders. As a result, the possibility of deadlock is reduced.

4. Suggest a redesign for the locking order to avoid future deadlocks.

The banking server should follow a consistent locking sequence. It should acquire the Account Database lock first, followed by the Customer Info Database lock and then the Transaction Logs lock, whenever these resources are required.

For example, a money transfer requiring A and B must acquire A before B. The system should release locks immediately after completing the required operations. It should also use database transactions and rollback procedures to handle failures safely. Lock timeouts can be used as an additional safeguard, although timeouts alone do not guarantee deadlock prevention.

Conclusion:

The banking server experiences deadlock because three transactions form a circular wait. Enforcing a consistent resource acquisition order prevents this type of deadlock and improves the reliability of banking operations.

QUESTION 3: MOBILE OS APP SWITCHING LAG (10 MARKS)

1. Analyze why the CPU remains idle despite many active processes.

The CPU may remain idle even when many applications are active because some processes are waiting for input/output operations, such as network responses, file access, or storage operations. These processes enter the blocked state and cannot execute until their I/O operations are completed. Therefore, having 20 active background applications does not necessarily mean that 20 processes are ready to use the CPU.

For example, WhatsApp may wait for a network response, while a music player may wait for audio data. If all processes are blocked and no process is ready to execute, the CPU may remain idle. The operating system must manage blocked processes efficiently and schedule ready processes whenever possible.

2. Determine whether the time quantum is helping or hurting performance.

The operating system uses Round Robin scheduling with a time quantum of 20 ms. Round Robin allocates a fixed time slice to each ready process in a circular order. A 20 ms quantum reduces context-switching overhead because each process can execute for a reasonable amount of time before switching to another process.

However, this quantum may reduce responsiveness when many processes are ready. An interactive application may have to wait for other processes to receive their time slices before getting CPU time. A large quantum also makes Round Robin behave more like FCFS scheduling, which may increase the perceived delay during app switching. Therefore, the 20 ms quantum may be less suitable for a highly interactive mobile environment.

3. Recommend a better scheduling method or quantum value.

Preemptive Priority Scheduling with dynamic priority adjustment is a suitable option for a mobile operating system. Foreground applications, such as WhatsApp, can receive higher priority because users expect them to respond quickly. Background applications can receive lower priority when the CPU is heavily loaded. The music player should also receive sufficient priority to maintain continuous playback.

If the system retains Round Robin scheduling, a time quantum of approximately 5 to 10 ms can be tested. A smaller quantum may improve responsiveness, but an excessively small quantum increases context-switching overhead. The operating system should select the value through performance testing.

4. Explain how the change will reduce switching lag and improve system throughput.

Priority-based scheduling reduces the time interactive applications wait before receiving CPU service. This makes application switching smoother and helps users perform tasks without unnecessary delays. Giving the music player sufficient CPU time also reduces the chance of audio interruptions caused by CPU scheduling delays.

Dynamic priority adjustment helps prevent background processes from being ignored indefinitely. A properly selected time quantum or priority policy can balance responsiveness and scheduling overhead. However, if the main bottleneck is slow network or storage access, scheduling changes alone may not completely eliminate the delay.

Conclusion:

Priority Scheduling with dynamic priority adjustment is suitable for mobile operating systems because it improves responsiveness while allowing background applications to continue executing. Appropriate tuning can reduce switching lag and improve the user experience.

QUESTION 4: DEADLOCK RISK IN AN OPERATING SYSTEM UPDATE (10 MARKS)

1. Identify whether this sequence can lead to a deadlock.

Yes, the given sequence can lead to deadlock because three update modules are waiting for resources held by one another. M1 copies update files and holds R1, which represents Disk I/O, while waiting for R2, the Kernel lock. M2 installs drivers and holds R2 while waiting for R3, the Configuration lock. M3 updates the registry and holds R3 while waiting for R1.

Since each module is waiting for another module to release a resource, none of the three modules can continue under the stated locking conditions. M4 performs system integrity validation, but no resource allocation or waiting relationship is provided for M4, so its involvement in the deadlock cannot be determined from the given information.

2. Explain the exact circular wait condition.

The resource allocation graph is:

R1 -> M1 -> R2 -> M2 -> R3 -> M3 -> R1

M1 waits for R2, which is held by M2. M2 waits for R3, which is held by M3. M3 waits for R1, which is held by M1. This forms a circular chain of waiting.

The four necessary conditions for deadlock are mutual exclusion, hold and wait, no preemption, and circular wait. These conditions are present under the assumptions that the resources are exclusive, modules retain their current resources while waiting, resources cannot be forcibly taken away, and the circular dependency continues.

3. Propose Banker's Algorithm-style resource allocation to avoid the risk.

Banker's Algorithm can be used to avoid deadlock by granting resource requests only when the resulting system state remains safe. The operating system must maintain information about the total resources, available resources, resources currently allocated to each module, and the maximum resource requirements of each module.

The remaining resource need is calculated as:

Need = Maximum - Allocation

When a module requests a resource, the OS checks whether the request is within its remaining need and whether sufficient resources are available. It then temporarily considers the allocation and performs a safety check. If there is an execution order in which all modules can finish and release their resources, the state is safe and the request may be granted. Otherwise, the request is delayed.

This approach requires reliable maximum resource requirements and an appropriate resource model. If module requirements are unknown or resources cannot be represented as reusable instances, other prevention techniques may be more practical.

4. Suggest how OS designers can prioritize modules to improve CPU utilization during the update.

OS designers should give suitable priority to time-sensitive update operations while allowing independent tasks to execute when another module is blocked. For example, an integrity validation task can run when its required files and resources are available, rather than forcing the CPU to wait for a blocked module.

The system should minimize the time resources remain locked, avoid performing lengthy operations while holding kernel locks, and schedule ready modules according to their dependencies. Priority should not override resource dependencies because a high-priority module cannot execute safely if a required resource is unavailable.

Conclusion:

The update modules can enter deadlock because they form a circular wait. Banker's Algorithm-style safety checking, efficient lock management, and dependency-aware scheduling can reduce deadlock risk and improve resource utilization.

QUESTION 5: CLOUD SERVER CPU UTILIZATION ANALYSIS (10 MARKS)

1. Calculate CPU utilization.

Given:

Probability of I/O waiting, p = 0.80 Number of virtual machines, n = 7

Using the formula:

CPU Utilization = 1 - p^n

Substituting the values:

CPU Utilization = 1 - (0.80)^7 CPU Utilization = 1 - 0.2097152 CPU Utilization = 0.7902848

Therefore:

CPU Utilization = 0.7902848 x 100 CPU Utilization = 79.03%

The estimated CPU utilization is approximately 79.03%.

2. Interpret whether the server is underutilized or optimally loaded.

According to the given formula, the server achieves approximately 79.03% CPU utilization. This indicates that the CPU is being used reasonably well under the assumptions of the model. It is not necessarily underutilized, although the value alone is insufficient to determine whether the server is optimally loaded.

Actual performance depends on the CPU demand of each VM, memory consumption, network delays, storage performance, and virtualization overhead. If the CPU frequently reaches maximum capacity and response times increase, the server may be overloaded even if average utilization appears acceptable. Therefore, CPU utilization should be evaluated together with latency, throughput, and resource contention.

3. Suggest how increasing or decreasing the number of VMs affects throughput.

Increasing the number of VMs can improve throughput because one VM may use the CPU while other VMs are waiting for network or disk operations. This allows the server to make better use of CPU resources that would otherwise remain idle.

However, adding too many VMs may increase memory pressure, CPU contention, and virtualization overhead. These problems can reduce throughput and increase response times. Decreasing the number of VMs may reduce contention and improve the performance of the remaining VMs, but it may also leave the CPU idle when fewer processes are ready.

Therefore, the cloud provider should determine the appropriate number of VMs by monitoring CPU utilization, memory usage, I/O waiting time, and response time.

4. Propose two OS-level strategies for improving CPU utilization.

The first strategy is efficient process scheduling. The operating system can use appropriate scheduling priorities to ensure that ready processes receive CPU time while other processes wait for I/O. This helps reduce unnecessary CPU idle time.

The second strategy is asynchronous I/O and efficient I/O scheduling. Asynchronous I/O allows applications to continue performing other work while network or disk operations are in progress. This reduces unnecessary blocking and helps the system handle more concurrent workloads.

Conclusion:

Using the given formula, CPU utilization is approximately 79.03%. Appropriate VM sizing, efficient scheduling, and asynchronous I/O can improve throughput while avoiding excessive resource contention.

QUESTION 6: HOSPITAL EMERGENCY SYSTEM PROCESS SCHEDULING (10 MARKS)

1. Identify the scheduling issue occurring in the system.

The main scheduling issue is starvation caused by priority scheduling. In the hospital information system, emergency patient registration (P1) has the highest priority, laboratory report generation (P2) has medium priority, and pharmacy billing (P3) has low priority.

During peak hours, emergency requests may continue to arrive. If the CPU always selects the highest-priority ready process, pharmacy billing may remain in the ready queue for a long time. This delays billing operations and can affect patients who need medicines or payment confirmation.

2. Recommend a suitable CPU scheduling algorithm.

Preemptive Priority Scheduling with aging is a suitable solution. Emergency registration receives high priority so that urgent patient requests can be handled quickly. Laboratory report generation and pharmacy billing can execute when higher-priority tasks do not require the CPU.

Preemptive scheduling allows an emergency process to interrupt a lower-priority process when necessary. Aging ensures that lower-priority processes receive opportunities to execute if they have waited for too long.

3. Explain how starvation can be prevented.

Starvation can be prevented by gradually increasing the priority of processes according to their waiting time. For example, if pharmacy billing remains in the ready queue for a long period, its priority can be increased until it receives CPU time.

The system can also use minimum service guarantees for non-emergency tasks and monitor waiting times. However, aging must be designed carefully so that a low-priority task does not delay a genuinely urgent emergency operation.

4. Discuss how the solution improves response time and fairness.

Priority Scheduling reduces the response time of emergency registration by allowing critical tasks to execute first. This is important in a hospital because delays in registering emergency patients can affect the delivery of medical services.

Aging improves fairness by ensuring that laboratory report generation and pharmacy billing do not remain blocked indefinitely. This allows the hospital system to handle both urgent and routine operations efficiently.

Conclusion:

Preemptive Priority Scheduling with aging is suitable for the hospital information system because it protects emergency response time while reducing starvation and improving fairness for lower-priority processes.

QUESTION 7: E-COMMERCE WAREHOUSE SYNCHRONIZATION (10 MARKS)

1. Identify the synchronization problem.

The main problem is a race condition caused by multiple worker threads accessing and modifying shared inventory data simultaneously. A race condition occurs when the result of a program depends on the order in which concurrent operations execute.

For example, suppose the available stock of a product is 10 units. Two worker threads process orders simultaneously. Both threads may read the stock value as 10 and independently calculate the remaining stock as 9. If both write the result, the final stock becomes 9 instead of 8. This causes an incorrect inventory count and may lead to overselling or duplicate order confirmations.

2. Explain why a critical section is required.

A critical section is the part of a program that accesses or modifies shared resources. In the warehouse system, checking stock availability, reducing the quantity, and confirming an order are critical operations because they modify shared inventory data.

Without synchronization, two or more threads may execute these operations at the same time. This can cause lost updates, incorrect stock counts, and inconsistent order records. Therefore, access to the shared inventory must be controlled so that conflicting operations cannot interfere with one another.

3. Recommend either semaphores or monitors to solve the issue.

A mutex or binary semaphore can be used to protect the inventory update critical section. The worker thread must acquire the lock before checking or modifying the stock and release it after completing the operation.

Pseudocode:

Initialize mutex = 1

Worker Thread: Lock(mutex) Check available stock

If stock > 0: Reduce stock by 1 Confirm order Else: Reject order

Unlock(mutex)

The lock ensures that only one thread at a time can execute the protected operation within the shared process. In a real system with multiple server instances, database transactions and atomic

updates must also be used because an application-level mutex does not protect the database from other processes.

4. Explain how the proposed solution maintains data consistency.

The mutex prevents multiple worker threads from simultaneously modifying the same shared inventory data within the protected process. This ensures that one operation completes its critical update before another conflicting operation begins.

Database transactions provide additional protection by ensuring that the stock update and order confirmation are handled consistently. If an operation fails, the transaction can be rolled back rather than leaving partially updated information.

Conclusion:

Using synchronization mechanisms such as mutexes or binary semaphores, together with atomic database transactions, prevents race conditions and improves the accuracy and consistency of warehouse inventory management.

QUESTION 8: VIDEO STREAMING SERVICE THREAD MANAGEMENT (10 MARKS)

1. Identify the thread-management problem.

The main problem is excessive thread creation and resource overhead. The streaming service creates one thread for every client connection. During a live sports event, 60,000 users connect simultaneously, causing the server to manage a very large number of threads.

Each thread consumes memory for its stack and other operating-system resources. A large number of active threads also increases context-switching overhead and scheduler workload. This can cause high CPU usage, increasing memory consumption, and frequent video buffering.

The reported CPU utilization of 95% indicates that the server is under heavy CPU load, although network bandwidth, encoding, and storage performance should also be checked to identify all causes of buffering.

2. Recommend a suitable multithreading model.

An event-driven architecture with a bounded worker-thread pool is suitable for the streaming service. Instead of creating one thread for every client, an event loop can monitor thousands of network connections using asynchronous I/O. A limited number of worker threads can process CPU-intensive operations.

The worker threads are reused instead of being created and destroyed for every request. This reduces resource overhead and allows the server to support many concurrent connections more efficiently.

3. Suggest two optimizations to improve scalability.

The first optimization is to implement a thread pool. The server should create a controlled number of worker threads based on available CPU resources. Incoming tasks can be placed in a queue and processed by existing threads. This reduces thread creation overhead and excessive context switching.

The second optimization is asynchronous I/O and connection multiplexing. Mechanisms such as epoll on Linux allow the server to monitor many network connections without dedicating a separate blocked thread to every client. The server can respond when a connection is ready for reading or writing.

4. Explain how the recommendations improve throughput.

The thread pool reduces memory usage and allows worker threads to be reused. Connection multiplexing reduces the number of threads required to manage simultaneous users. As a result, the server spends fewer resources on thread management and can devote more resources to processing requests and transmitting video data.

These improvements increase scalability and can reduce buffering caused by CPU and memory pressure. The server should also use load balancing and content delivery networks when necessary to distribute traffic and reduce the load on a single server.

Conclusion:

An event-driven architecture with a bounded worker-thread pool is more scalable than the one-thread-per-client approach. It reduces resource overhead and improves the server's ability to handle thousands of concurrent users.

QUESTION 9: SMART TRAFFIC CONTROL DEADLOCK (10 MARKS)

1. Determine whether a deadlock exists.

Yes, the given situation contains a deadlock, assuming each junction is an exclusive resource and a transaction cannot release its current junction until it obtains the next one.

T1 holds R1 and requests R2. T2 holds R2 and requests R3. T3 holds R3 and requests R4. T4 holds R4 and requests R1. Every transaction is waiting for a resource held by another transaction.

The resource allocation graph is:

R1 -> T1 -> R2 -> T2 -> R3 -> T3 -> R4 -> T4 -> R1

This forms a circular dependency in which no transaction can proceed without the release of another resource in the cycle.

2. Explain the four necessary conditions for deadlock in this scenario.

The first condition is mutual exclusion. Each junction resource is assumed to be exclusively allocated to one conflicting transaction at a time.

The second condition is hold and wait. Each transaction holds one junction while requesting another junction.

The third condition is no preemption. A junction cannot be forcibly taken away from a transaction under the assumed resource-locking model.

The fourth condition is circular wait. T1 waits for R2, T2 waits for R3, T3 waits for R4, and T4 waits for R1. These dependencies form a closed cycle.

When all four conditions exist together, deadlock can occur. In a real traffic-control system, physical vehicle occupancy and safety rules must also be considered before concluding that a software resource can be released.

3. Recommend one recovery technique.

Deadlock recovery through transaction rollback and resource release is a suitable technique. The controller first detects the circular wait and selects a transaction to cancel or roll back according to its priority and recovery cost.

The selected transaction releases its reserved resources, allowing another waiting transaction to proceed. The cancelled transaction can be restarted when the required resources become available. In an actual traffic system, recovery must use safe traffic-control procedures rather than simply releasing a junction that is physically occupied.

4. Suggest a redesign that prevents similar deadlocks.

The system should use a centralized traffic coordinator to control access to junctions. Before a vehicle enters a conflicting area, the coordinator should reserve the required junctions in a predefined order or ensure that the next junction is available.

If resources are locked, every transaction should follow a consistent global acquisition order. A transaction that cannot obtain the next resource should wait without holding a conflicting set of resources that could create circular wait. The controller should also monitor congestion and provide safe alternative routes when possible.

Conclusion:

The cyclic dependency among the four junctions creates deadlock under the stated assumptions. Centralized resource reservation, consistent acquisition order, and safe recovery procedures can prevent the system from becoming permanently blocked.

QUESTION 10: UNIVERSITY EXAMINATION SERVER RESOURCE ALLOCATION (10 MARKS)

1. Explain how Banker's Algorithm can be applied.

Banker's Algorithm is a deadlock-avoidance technique that grants resource requests only when the resulting system state remains safe. In the university examination server, the resources include CPU capacity, memory, and database connections required by student sessions.

Thousands of students may log in simultaneously and request resources. The operating system or resource manager can maintain information about available resources, resources allocated to each session, and the maximum resources each session may require.

When a new request arrives, the system checks whether sufficient resources are available and whether granting the request would leave the system in a safe state. If the resulting state is safe, the request can be granted. Otherwise, the request must wait until resources become available or the state changes.

2. Describe the information required before granting a request.

- Banker's Algorithm requires four main data structures.
- Available represents the number of currently available instances of each resource.
- Maximum represents the maximum resource requirement of each active session.
- Allocation represents the resources currently allocated to each session.
- Need represents the remaining resources required by each session to complete execution.
- The remaining need is calculated using the formula:
- Need = Maximum - Allocation

Before granting a request, the system checks that the request does not exceed the session's remaining need and that the requested resources are available. It then performs a safety check before making the allocation permanent.

3. Explain how the system determines whether it is in a safe state.

A system is in a safe state if there is at least one sequence in which every active session can obtain its remaining required resources, finish execution, and release its allocated resources.

The safety algorithm starts by setting Work equal to Available. It then searches for a session whose remaining Need is less than or equal to Work. If such a session exists, the algorithm assumes that the session finishes and releases its allocated resources.

The available working resources are updated as follows:

Work = Work + Allocation

The process is repeated until all sessions can finish or no additional session can be selected. If all sessions can finish in some order, the system is in a safe state. If the algorithm cannot find a sequence that allows all sessions to finish, the state is unsafe, and the proposed request should not be granted under Banker's Algorithm.

4. Discuss one limitation of Banker's Algorithm in large-scale cloud systems.

One major limitation is that Banker's Algorithm requires reliable knowledge of the maximum resource requirements of each session in advance. In a large cloud examination system, thousands of students may log in and log out continuously, and their resource requirements may change during the examination.

Maintaining accurate resource information and repeatedly performing safety checks can introduce computational overhead. The algorithm may also delay resource requests even when some resources are currently available, because granting them might lead to an unsafe state.

Furthermore, CPU scheduling, memory allocation, and database connection management may be handled by different components. Applying one centralized Banker’s Algorithm across all these resources can be difficult.

Conclusion:

Banker's Algorithm can help prevent deadlocks by checking the safety of every proposed resource allocation. However, its requirement for known maximum resource demands and its computational overhead can limit its direct use in large-scale cloud examination systems.

RUBRICS FOR CALCULATION

Scenario based assessment

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding ; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand